

Assignment 2: Fire-spread model in NetLogo

Maitreyi Chakrapani (820264599)

2026-08-20

Part A: Build the model

Paste your complete NetLogo code (the `globals`, `setup`, and `go` procedures):

```
globals [
  initial-trees
  burned-trees
]

to setup
  clear-all
  ask patches [
    if random 100 < density [ set pcolor green ]
  ]
  ask patches with [ pxcor = min-pxcor ] [
    if pcolor = green [ set pcolor red ]
  ]
  set initial-trees count patches with [ pcolor = green or pcolor = red ]
  set burned-trees 0
  reset-ticks
end

to go
  if not any? patches with [ pcolor = red ] [ stop ]
  ask patches with [ pcolor = red ] [
    ask neighbors4 with [ pcolor = green ] [
      set pcolor red
    ]
  ]
  set pcolor grey
  set burned-trees burned-trees + 1
end
```

```
]
tick
end
```

Task A1. Embed your screenshot of the final state of the world at `density = 60`:

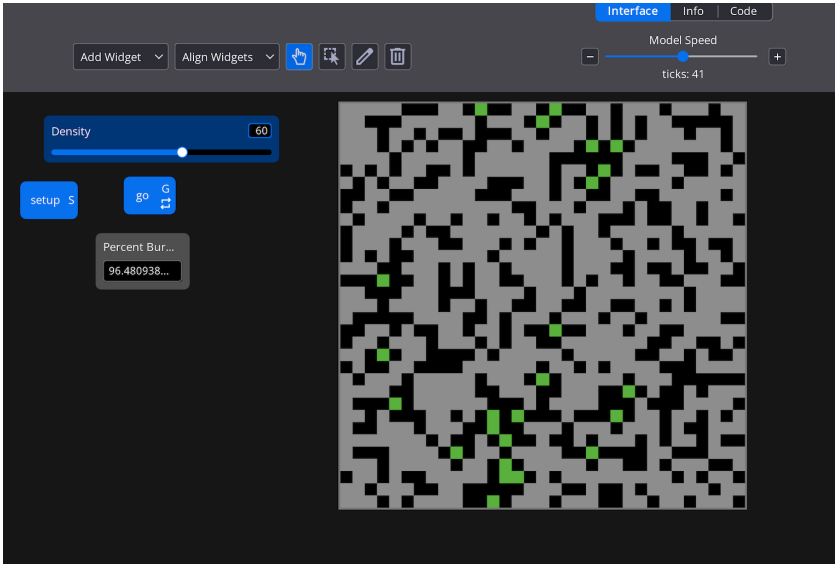


Figure 1: Final state of the environment at density 60

Task A2. Percent burned at the end of the run: *96.48%*

Part B: Find the critical density

Task B1. Complete the table: ten runs at each density, percent burned each time.

Den- sity	Run 1	Run 2	Run 3	Run 4	Run 5	Run 6	Run 7	Run 8	Run 9	Run 10	Mean
40	27.4	10.5	7.5	12.1	7.5	16.3	19.7	8.8	11.4	22.9	14.41
50	46.4	64	13.4	28.3	48.8	30.4	27.1	26.45	27.42	43.64	35.591
55	81	66.39	68.2	67.1	55.36	38.1	86.44	82.25	75	28.23	64.807
58	78.2	65.95	93.15	64.8	69.2	86.49	81.7	84.1	53.5	73.75	75.084
59	78.35	81.17	93.75	72.5	56	81.19	90.53	93.11	83.6	90.21	82.041
60	87.3	79	91	92.69	79.45	74.5	90.1	92.4	86.1	84.69	85.723
62	96.3	94.86	86.5	94.57	95	89.1	90.9	92.86	96.2	96.96	93.325
65	98.9	96.4	97.4	95.68	97.79	96.79	94	97.15	96.4	96.07	96.658

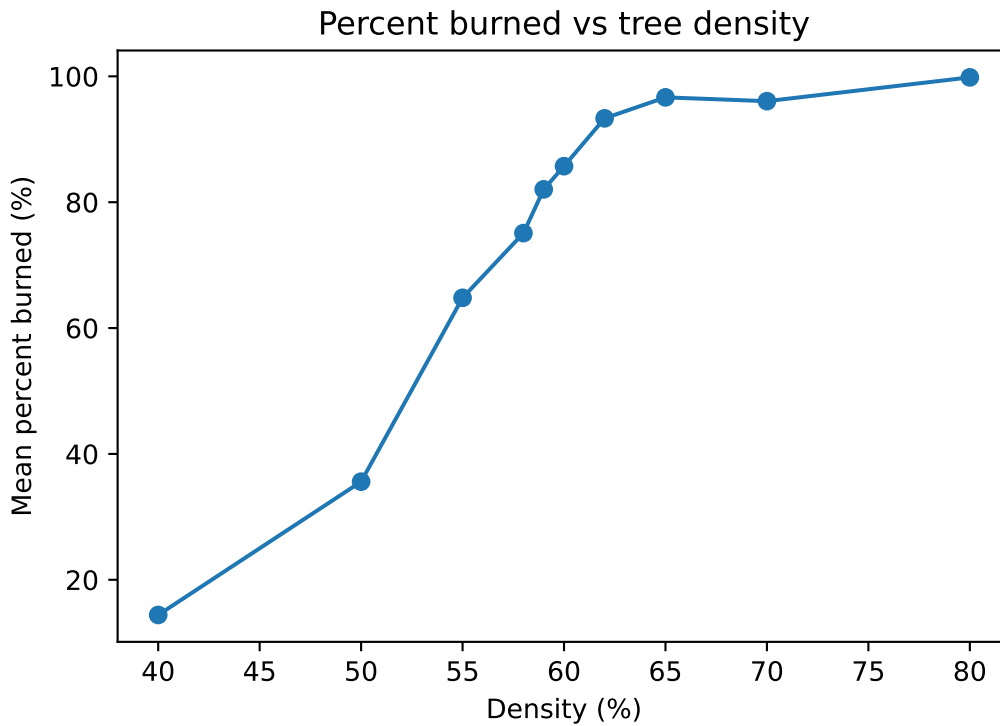
Den- sity	Run 1	Run 2	Run 3	Run 4	Run 5	Run 6	Run 7	Run 8	Run 9	Run 10	Mean
70	98.5	94.26	96.96	99	94.3	95.5	96.88	93.57	94.1	97.47	96.054
80	99.88	100	99.88	99.77	99.88	99.88	99.29	99.88	99.88	100	99.834

Task B2. Plot mean percent burned vs density. By hand or Excel: embed a photo or screenshot. In Python: use the code cell below.

```
import matplotlib.pyplot as plt

density = [40, 50, 55, 58, 59, 60, 62, 65, 70, 80]
mean_burned = [14.41, 35.591, 64.807, 75.084, 82.041, 85.723, 93.325, 96.658, 96.054, 99.834]

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(density, mean_burned, marker="o")
ax.set_xlabel("Density (%)")
ax.set_ylabel("Mean percent burned (%)")
ax.set_title("Percent burned vs tree density")
plt.show()
```



Task B3 (critical-density estimate). I can estimate the critical density to be ~55%. This is the lowest density at which the mean percent burned exceeds 50%, indicating that the fire can spread through a significant portion of the forest. Below this density, the fire tends to die out quickly, while above it, the fire spreads more extensively.

Looking at the mean percent burned, the biggest jump happens between density 50 (35.6%) and density 55 (64.8%), a jump of ~29 points, the largest single-step increase in the whole dataset. After that, values climb more gradually and start leveling off by 62–65%. So my estimate for the critical density is around 55%. This is reasonably close to the theoretical value of ~59%, albeit a bit lower. However, I have only done 10 runs per density and the stochastic nature of the model means that more runs would give a better estimate. Overall, my estimate is in the right ballpark and captures the sharp transition in fire spread behavior.

Task B4 (variability near the threshold). At density 40, the 10 runs range from about 7.5 to 27.4, which are consistently low values. At density 80, runs range from 99.3 to 100, very high values. At density 55, runs range from 28.2 all the way up to 86.4, showing a wide spread of outcomes. This variability is expected, as at low density there are not enough trees to sustain the fire, while at high density, the fire can easily spread. Near the critical density, small differences in tree placement can lead to very different outcomes, which is why we see such a wide range of burned percentages at density 55.

Part C: Summary ODD documentation

The overall *purpose* of our model is to understand how the connectivity of a randomly distributed resource (trees) affects the large-scale spread of a diffusive process (fire) across a landscape. **Specifically, we are addressing the following questions:** how does the total area burned respond to tree density, and does this response show a sharp threshold (critical density) rather than a smooth, gradual increase? **To consider our model realistic enough for its purpose, we use the following *patterns*:** a sharp, non-linear jump in percent burned as density increases (rather than a straight-line relationship), and high run-to-run variability in outcomes specifically near the threshold density, both of which are hallmarks of percolation behavior seen in real wildfire spread.

The model includes the following *entities*: patches, which represent individual cells of forest land that are either occupied by a tree or empty. **The *state variables* characterising these entities are listed in Table 1** (patch color: green = unburned tree, red = burning tree, grey = burned-out tree, black/default = no tree; and patch coordinates pxcor, pycor). **The *spatial and temporal resolution and extent* are** the standard NetLogo patch grid, where each patch represents a single tree location (occupied or not) and each tick represents one discrete time step of fire spreading to adjacent trees; a run ends when no patches remain red (no more burning trees), i.e. when the fire has stopped spreading.

The most important *processes* of the model, which are repeated every *time step*, are, in order: (1) spread - every currently burning (red) patch ignites any green neighbor

among its four von Neumann neighbors (`neighbors4`), turning them red; (2) burnout - each patch that was burning at the start of the tick turns grey and is counted toward burned-trees. These two processes happen simultaneously for all burning patches each tick, and the simulation halts once no red patches remain.

The most important *design concepts* of the model are stochasticity, since tree placement is randomised (each patch becomes a tree with probability equal to density), meaning that even identical density settings can produce very different outcomes from run to run. This randomness is what generates the variability observed near the critical density.

The model is initialised with trees placed randomly across the grid, where each patch independently becomes green (a tree) with probability equal to the density slider value (density out of 100); fire is then started by setting every tree patch in the leftmost column (`pxcor` = `min-pxcor`) to red (burning); `initial-trees` is set to the count of all tree/burning patches, `burned-trees` is reset to 0, and the tick counter is reset to begin the simulation.

Table 1. Entities and state variables of the model.

En- tity	Vari- able	Description	Possible values	Units
Patch	<code>pcolor</code>	The state of a patch: whether it holds an unburned tree, a burning tree, a burned-out tree, or no tree at all	black (no tree), green (unburned tree), red (burning tree), grey (burned tree)	categorical (color)
Patch	<code>pxcor</code>	The x-coordinate of a patch, used to determine which patches are on the left edge where fire starts	integers from <code>min-pxcor</code> to <code>max-pxcor</code>	patches
Patch	<code>pycor</code>	The y-coordinate of a patch	integers from <code>min-pycor</code> to <code>max-pycor</code>	patches
Global	<code>density</code>	Slider controlling the probability that any given patch is initially occupied by a tree	0 to 100	percent (%)
Global	<code>initial-trees</code>	The total number of tree patches (green + red) present at the start of the run, i.e. before any burning has occurred	0 to total number of patches	count (patches)
Global	<code>burned-trees</code>	Running count of the number of patches that have burned (turned from red to grey) so far	0 to <code>initial-trees</code>	count (patches)
Global	<code>ticks</code>	The model's internal clock, incremented once per time step; used to track how long the fire takes to stop spreading	0, 1, 2, ...	ticks (time steps)